

Numerical Methods Assignment 2 & 3:

Team Members :

ES24BTECH11018	Malhar Kanhegaonkar
ES24BTECH11021	Kripalu Sonar
ES24BTECH11036	Taleem Hossain

Assignment 2

1. Harshad Numbers

A:

We implemented an algorithm that calculates a number's factorial, sums its digits, and checks if the factorial is a Harshad number. The program runs efficiently for starting inputs up to around **100,000**.

B:

For finding **n consecutive Harshad numbers**, our algorithm has a time complexity of **$O(N)$** , where N is the first number in the sequence of n consecutive Harshads. It performs well for **$n \leq 10$** , but becomes impractical for **$n = 11$** , where N reaches around **15 digits**. On our laptops, that would take nearly **4 months** to compute.

We explored advanced mathematical methods that significantly reduce the search space, but the proofs behind them were complex and not fully understood. Hence, we proceeded with our brute-force approach.

For the **non-existence of more than 20 consecutive Harshads**, we referred to **Cooper and Kennedy's original paper**:

<https://www.fq.math.ca/Scanned/31-2/cooper.pdf>

2. Modified Legendre Polynomials

A:

Using Python's `numpy.legendre` library and the recursive formula, we obtained nearly identical coefficients for the modified Legendre polynomials.

B:

We used the coefficients from part A to construct the polynomials.

C:

For large matrices, numerical instability appeared. To fix this, we converted the **companion matrix** into a **Jacobi (tridiagonal) matrix** using the **Golub-Welsh method**, which is stable and has the same eigenvalues. These eigenvalues correspond to the roots.

D:

We used an LU decomposition algorithm with pivoting (to prevent zeros on the diagonals), followed by forward and backward substitution to compute the solution vector $\mathbf{x}$. We manually verified results for $\mathbf{n} = 2$.

E:

Finally, we used the **Newton–Raphson method**, taking advantage of the fact that all roots of modified Legendre polynomials lie between **0 and 1**. Thus, initial guesses of **0** and **1** work well for the smallest and largest roots respectively.

Assignment 3

1. Golub–Welsh Method

We computed the **Jacobi matrix** as given, used NumPy's eigenvalue function to find the roots, then calculated the corresponding **weights**. We plotted the relationship between the roots and their weights.

The **A** and **B** matrices were derived following the class notes.

2. Orthogonal Collocation

We solved the given ODE using the **A** and **B** matrices obtained from the Golub–Welsh method and modified Legendre polynomials.

The ODE was defined for $\eta \in [0, \infty)$, so we transformed it to $\mathbf{z} \in [0, 1]$ using the substitution $\mathbf{z} = -\ln(\eta)$. The resulting ODE in $\mathbf{z}$ was then solved by forming **n equations** in y_i using A and B , and solving for y_i .

From $\mathbf{y} = \mathbf{Qd}$, we determined $\mathbf{d}$, and reconstructed $\mathbf{y} = \mathbf{d}_0 + \mathbf{d}_1\mathbf{x} + \mathbf{d}_2\mathbf{x}^2 + \dots + \mathbf{d}_n\mathbf{x}^n$.

Finally, using $\mathbf{z} = \exp(-\eta)$, we plotted the **Numerical vs Analytical** solutions with:

- **T** on the y-axis,
- **x (rod length)** on the x-axis, and
- a **time slider** to visualize changes at different time instants.

We also generated an **error field** matrix showing (*Analytical* – *Numerical*) errors across $\mathbf{x}$ and $\mathbf{t}$.

GUI

We faced challenges integrating the backend and frontend, as performance slowed when running computations from the GUI.

The interface includes:

- Aesthetic design with glass and fluid hover effects,
- Animated progress tracking,
- A completion screen displaying the final results.

Below are the final GUI visuals:

